JAVA STRING FUNCTIONS – 15 PROGRAMS

Complete Java Lab Record

1. length()

1. Aim

To write a Java program to find the number of characters present in a String using the length() method.

2. Definition

The length() method returns the total number of characters present in a String.

Syntax

```
string.length();
```

3. Structure

```
String
  ↓
length()
  ↓
Result
```

4. Program

```
class StringLength {
    public static void main(String[] args) {

        String str = "Hello Java";

        int length = str.length();

        System.out.println("String: " + str);
        System.out.println("Length of String: " + length);
    }
}
```

5. Sample Output

```
String: Hello Java
Length of String: 10
```

6. Test Case Verification

Test Case	Input	Expected Output
1	Hello Java	10
2	Java	4
3	Hello	5
4	Computer	8

7. Conclusion

Thus, the Java program successfully finds the number of characters in a String using the length() method.

2. charAt()

1. Aim

To write a Java program to retrieve a character from a String at a specified index using the charAt() method.

2. Definition

The charAt() method returns the character present at the specified index of a String. String indexing starts from 0.

Syntax

```
string.charAt(index);
```

3. Structure

```
String
  ↓
charAt()
  ↓
Result
```

4. Program

```
class CharAtExample {
    public static void main(String[] args) {

        String str = "Hello Java";

        char ch = str.charAt(6);

        System.out.println("String: " + str);
        System.out.println("Character at index 6: " + ch);
    }
}
```

5. Sample Output

```
String: Hello Java
Character at index 6: J
```

6. Test Case Verification

Test Case	String	Index	Expected Output
1	Hello	1	e
2	Java	0	J
3	Computer	3	p

7. Conclusion

Thus, the charAt() method successfully retrieves a character from the specified index of a String.

3. toUpperCase()

1. Aim

To write a Java program to convert a String into uppercase using the toUpperCase() method.

2. Definition

The toUpperCase() method converts lowercase letters in a String into uppercase letters.

Syntax

```
string.toUpperCase();
```

3. Structure

```
String
  ↓
toUpperCase()
  ↓
Result
```

4. Program

```
class UpperCaseExample {
    public static void main(String[] args) {

        String str = "hello java";

        String result = str.toUpperCase();

        System.out.println("Original String: " + str);
        System.out.println("Uppercase String: " + result);
    }
}
```

5. Sample Output

```
Original String: hello java
Uppercase String: HELLO JAVA
```

6. Test Case Verification

Test Case	Input	Expected Output
1	hello	HELLO
2	java programming	JAVA PROGRAMMING
3	computer	COMPUTER

7. Conclusion

Thus, the toUpperCase() method successfully converts the String into uppercase.

4. toLowerCase()

1. Aim

To write a Java program to convert a String into lowercase using the toLowerCase() method.

2. Definition

The toLowerCase() method converts uppercase letters in a String into lowercase letters.

Syntax

```
string.toLowerCase();
```

3. Structure

```
String
  ↓
toLowerCase()
  ↓
Result
```

4. Program

```
class LowerCaseExample {
    public static void main(String[] args) {

        String str = "HELLO JAVA";

        String result = str.toLowerCase();

        System.out.println("Original String: " + str);
        System.out.println("Lowercase String: " + result);
    }
}
```

5. Sample Output

```
Original String: HELLO JAVA
Lowercase String: hello java
```

6. Test Case Verification

Test Case	Input	Expected Output
1	HELLO	hello
2	JAVA PROGRAMMING	java programming
3	COMPUTER	computer

7. Conclusion

Thus, the toLowerCase() method successfully converts the String into lowercase.

5. equals()

1. Aim

To write a Java program to compare two Strings using the equals() method.

2. Definition

The equals() method compares the contents of two Strings. It returns true if both Strings have the same contents; otherwise, it returns false.

Syntax

```
string1.equals(string2);
```

3. Structure

```
String
  ↓
equals()
  ↓
Result
```

4. Program

```
class EqualsExample {
    public static void main(String[] args) {

        String str1 = "Java";
        String str2 = "Java";

        boolean result = str1.equals(str2);

        System.out.println("String 1: " + str1);
        System.out.println("String 2: " + str2);
        System.out.println("Strings are equal: " + result);
    }
}
```

5. Sample Output

```
String 1: Java
String 2: Java
Strings are equal: true
```

6. Test Case Verification

Test Case	String 1	String 2	Expected Output
1	Java	Java	true
2	Java	java	false
3	Hello	World	false

7. Conclusion

Thus, the equals() method successfully compares the contents of two Strings.

6. equalsIgnoreCase()

1. Aim

To write a Java program to compare two Strings without considering their letter case using equalsIgnoreCase().

2. Definition

The equalsIgnoreCase() method compares two Strings while ignoring differences between uppercase and lowercase letters.

Syntax

```
string1.equalsIgnoreCase(string2);
```

3. Structure

```
String
  ↓
equalsIgnoreCase()
  ↓
Result
```

4. Program

```
class EqualsIgnoreCaseExample {
    public static void main(String[] args) {

        String str1 = "Java";
        String str2 = "JAVA";

        boolean result = str1.equalsIgnoreCase(str2);

        System.out.println("String 1: " + str1);
        System.out.println("String 2: " + str2);
        System.out.println("Strings are equal: " + result);
    }
}
```

5. Sample Output

```
String 1: Java
String 2: JAVA
Strings are equal: true
```

6. Test Case Verification

Test Case	String 1	String 2	Expected Output
1	Java	JAVA	true
2	Hello	HELLO	true
3	Java	Python	false

7. Conclusion

Thus, the equalsIgnoreCase() method successfully compares two Strings without considering their case.

7. concat()

1. Aim

To write a Java program to join two Strings using the concat() method.

2. Definition

The concat() method combines one String with another String and returns the resulting String.

Syntax

```
string1.concat(string2);
```

3. Structure

```
String
  ↓
concat()
  ↓
Result
```

4. Program

```
class ConcatExample {
    public static void main(String[] args) {

        String str1 = "Hello ";
        String str2 = "Java";

        String result = str1.concat(str2);

        System.out.println("First String: " + str1);
        System.out.println("Second String: " + str2);
        System.out.println("Combined String: " + result);
    }
}
```

5. Sample Output

```
First String: Hello
Second String: Java
Combined String: Hello Java
```

6. Test Case Verification

Test Case	String 1	String 2	Expected Output
1	Hello	Java	Hello Java
2	Good	Morning	Good Morning
3	Computer	Science	Computer Science

7. Conclusion

Thus, the concat() method successfully joins two Strings.

8. substring()

1. Aim

To write a Java program to extract a portion of a String using the substring() method.

2. Definition

The substring() method returns a part of a String starting from the specified index.

Syntax

```
string.substring(startIndex);
```

3. Structure

```
String
  ↓
substring()
  ↓
Result
```

4. Program

```
class SubstringExample {
    public static void main(String[] args) {

        String str = "Programming";

        String result = str.substring(3);

        System.out.println("Original String: " + str);
        System.out.println("Substring: " + result);
    }
}
```

5. Sample Output

```
Original String: Programming
Substring: gramming
```

6. Test Case Verification

Test Case	String	Starting Index	Expected Output
1	Programming	3	gramming
2	Computer	4	uter
3	Hello	2	llo

7. Conclusion

Thus, the substring() method successfully extracts a portion of the String from the specified index.

9. indexOf()

1. Aim

To write a Java program to find the first occurrence of a character in a String using indexOf().

2. Definition

The indexOf() method returns the index of the first occurrence of the specified character or String.

Syntax

```
string.indexOf(character);
```

3. Structure

```
String
  ↓
indexOf()
  ↓
Result
```

4. Program

```
class IndexOfExample {
    public static void main(String[] args) {

        String str = "Hello";

        int index = str.indexOf('l');

        System.out.println("String: " + str);
        System.out.println("Index of 'l': " + index);
    }
}
```

5. Sample Output

```
String: Hello
Index of 'l': 2
```

6. Test Case Verification

Test Case	String	Character	Expected Output
1	Hello	l	2
2	Java	a	1
3	Computer	o	1

7. Conclusion

Thus, the indexOf() method successfully finds the first occurrence of the specified character.

10. lastIndexOf()

1. Aim

To write a Java program to find the last occurrence of a character in a String using lastIndexOf().

2. Definition

The lastIndexOf() method returns the index of the last occurrence of the specified character or String.

Syntax

```
string.lastIndexOf(character);
```

3. Structure

```
String
  ↓
lastIndexOf()
  ↓
Result
```

4. Program

```
class LastIndexOfExample {
    public static void main(String[] args) {

        String str = "Hello";

        int index = str.lastIndexOf('l');

        System.out.println("String: " + str);
        System.out.println("Last index of 'l': " + index);
    }
}
```

5. Sample Output

```
String: Hello
Last index of 'l': 3
```

6. Test Case Verification

Test Case	String	Character	Expected Output
1	Hello	l	3
2	Java	a	3
3	Banana	a	5

7. Conclusion

Thus, the lastIndexOf() method successfully finds the last occurrence of the specified character.

11. replace()

1. Aim

To write a Java program to replace characters in a String using the replace() method.

2. Definition

The replace() method replaces every occurrence of a specified character with another character.

Syntax

```
string.replace(oldCharacter, newCharacter);
```

3. Structure

```
String
  ↓
replace()
  ↓
Result
```

4. Program

```
class ReplaceExample {
    public static void main(String[] args) {

        String str = "Hello";

        String result = str.replace('l', 'p');

        System.out.println("Original String: " + str);
        System.out.println("After Replacement: " + result);
    }
}
```

5. Sample Output

```
Original String: Hello
After Replacement: Heppo
```

6. Test Case Verification

Test Case	String	Replace	Expected Output
1	Hello	l → p	Heppo
2	Java	a → o	Jovo
3	Apple	p → t	Attle

7. Conclusion

Thus, the replace() method successfully replaces the specified characters in a String.

12. trim()

1. Aim

To write a Java program to remove leading and trailing spaces from a String using the trim() method.

2. Definition

The trim() method removes leading and trailing spaces from a String.

Syntax

```
string.trim();
```

3. Structure

```
String
  ↓
trim()
  ↓
Result
```

4. Program

```
class TrimExample {
    public static void main(String[] args) {

        String str = "   Hello Java   ";

        String result = str.trim();

        System.out.println("Original String: " + str);
        System.out.println("After trim(): " + result);
    }
}
```

5. Sample Output

```
Original String:    Hello Java
After trim(): Hello Java
```

6. Test Case Verification

Test Case	Input	Expected Output
1	Hello	Hello
2	Java	Java
3	Computer Science	Computer Science

7. Conclusion

Thus, the trim() method successfully removes leading and trailing spaces from the String.

13. contains()

1. Aim

To write a Java program to check whether a String contains a specified sequence of characters using contains().

2. Definition

The contains() method checks whether the specified sequence of characters is present in the String. It returns true or false.

Syntax

```
string.contains(sequence);
```

3. Structure

```
String
  ↓
contains()
  ↓
Result
```

4. Program

```
class ContainsExample {
    public static void main(String[] args) {

        String str = "I love Java";

        boolean result = str.contains("Java");

        System.out.println("String: " + str);
        System.out.println("Contains Java: " + result);
    }
}
```

5. Sample Output

```
String: I love Java
Contains Java: true
```

6. Test Case Verification

Test Case	String	Search Text	Expected Output
1	I love Java	Java	true
2	Hello World	Java	false
3	Computer Science	Science	true

7. Conclusion

Thus, the contains() method successfully checks whether the specified text is present in a String.

14. startsWith()

1. Aim

To write a Java program to check whether a String starts with a specified prefix using startsWith().

2. Definition

The startsWith() method checks whether a String begins with the specified sequence of characters.

Syntax

```
string.startsWith(prefix);
```

3. Structure

```
String
  ↓
startsWith()
  ↓
Result
```

4. Program

```
class StartsWithExample {
    public static void main(String[] args) {

        String str = "Java Programming";

        boolean result = str.startsWith("Java");

        System.out.println("String: " + str);
        System.out.println("Starts with Java: " + result);
    }
}
```

5. Sample Output

```
String: Java Programming
Starts with Java: true
```

6. Test Case Verification

Test Case	String	Prefix	Expected Output
1	Java Programming	Java	true
2	Hello World	Hello	true
3	Java Programming	Python	false

7. Conclusion

Thus, the startsWith() method successfully checks whether a String begins with the specified prefix.

15. endsWith()

1. Aim

To write a Java program to check whether a String ends with a specified suffix using endsWith().

2. Definition

The endsWith() method checks whether a String ends with the specified sequence of characters.

Syntax

```
string.endsWith(suffix);
```

3. Structure

```
String
  ↓
endsWith()
  ↓
Result
```

4. Program

```
class EndsWithExample {
    public static void main(String[] args) {

        String str = "Java Programming";

        boolean result = str.endsWith("Programming");

        System.out.println("String: " + str);
        System.out.println("Ends with Programming: " + result);
    }
}
```

5. Sample Output

```
String: Java Programming
Ends with Programming: true
```

6. Test Case Verification

Test Case	String	Suffix	Expected Output
1	Java Programming	Programming	true
2	Hello World	World	true
3	Java Programming	Java	false

7. Conclusion

Thus, the endsWith() method successfully checks whether a String ends with the specified suffix.

QUICK REVISION TABLE

No.	Function	Purpose
1	length()	Finds number of characters
2	charAt()	Gets character at an index
3	toUpperCase()	Converts to uppercase
4	toLowerCase()	Converts to lowercase
5	equals()	Compares String contents
6	equalsIgnoreCase()	Compares without considering case
7	concat()	Joins two Strings
8	substring()	Extracts part of a String
9	indexOf()	Finds first occurrence
10	lastIndexOf()	Finds last occurrence
11	replace()	Replaces characters
12	trim()	Removes leading/trailing spaces
13	contains()	Checks whether text exists
14	startsWith()	Checks beginning of String
15	endsWith()	Checks ending of String